Informe

Entrega 1: Aplicación Web de Gestión de Vídeos

Asignatura: ISDCM

Autores: Arthur Bohin y Zhehan Xiang

Fecha: 23 de marzo de 2026

Índice

1. Introducción **2**

2. Arquitectura — Patrón MVC **2**
2.1. Modelo (`model`) 2
2.2. Controlador (`controller`) 3
2.3. Vista (`view`) 4

3. Gestión de Sesiones **4**

4. Gestión de la Base de Datos **5**
4.1. Esquema de tablas 5
4.2. Prevención de inyección SQL 5
4.3. Gestión de conexiones 6

5. Uso de Packages **6**

6. Control de Errores y Validación de Entrada **6**
6.1. Validación en el lado del cliente (HTML) 6
6.2. Validación en el lado del servidor 6
6.3. Control de errores en redirecciones 7
6.4. Control de duplicados 7

7. Configuración del Despliegue **7**

8. Flujo de Navegación de la Aplicación **8**

9. Conclusiones **8**

1 Introducción

El presente informe describe el diseño, la implementación y las decisiones técnicas adoptadas en el desarrollo del **Entrega 1**, cuyo objetivo es construir una aplicación web de gestión de vídeos. La aplicación permite a los usuarios registrarse, autenticarse y, una vez autenticados, registrar vídeos y consultar el listado completo de vídeos almacenados en la base de datos.

El proyecto sigue el patrón de arquitectura **Modelo-Vista-Controlador (MVC)** y se despliega sobre un servidor de aplicaciones compatible con **Jakarta EE 9.1**. La persistencia de datos se gestiona mediante **Apache Derby** como sistema gestor de bases de datos relacional.

2 Arquitectura — Patrón MVC

La aplicación se estructura siguiendo estrictamente el patrón **Modelo-Vista-Controlador**, organizando el código en tres paquetes Java y un directorio de vistas JSP:

Estructura del proyecto

```
src/main/java/
+-- controlador/        <- Controladores (Servlets)
|   +-- ServletUsuarios.java
|   +-- ServletVideos.java
+-- model/              <- Modelo (acceso a datos y
|   |                      logica de negocio)
|   +-- DBConnection.java
|   +-- Usuario.java
|   +-- Video.java
src/main/webapp/
+-- index.jsp           <- Pagina de entrada
|                          (redireccion a Login)
+-- WEB-INF/
|   +-- web.xml
+-- vista/              <- Vistas (JSP)
|   +-- login.jsp
|   +-- registroUsu.jsp
|   +-- registroVid.jsp
|   +-- listaVideos.jsp
|   +-- resultado.jsp
```

2.1. Modelo (`model`)

El paquete `model` encapsula toda la lógica de acceso a la base de datos. Ninguna clase del controlador ni de la vista ejecuta consultas SQL directamente; toda interacción con la base de datos se delega en las clases del modelo.

- `DBConnection.java`: Clase utilitaria que centraliza la obtención de conexiones JDBC a la base de datos Apache Derby. Carga el driver `org.apache.derby.jdbc.ClientDriver`

y establece la conexión con la URL `jdbc:derby://localhost:1527/pr2` y las credenciales correspondientes.

- `Usuario.java`: Gestiona las operaciones relativas a los usuarios:
  - `registrar(nombre, apellidos, email, username, password)`: Inserta un nuevo usuario en la tabla `usuarios`, verificando previamente que no exista un registro con el mismo `username` (control de duplicados).
  - `login(username, password)`: Verifica las credenciales del usuario consultando la tabla `usuarios` mediante `PreparedStatement`, retornando `true` si la combinación usuario/contraseña es válida.
- `Video.java`: Gestiona las operaciones relativas a los vídeos:
  - `registrar(titulo, autor, fecha, duracion, descripcion, formato)`: Inserta un nuevo vídeo en la tabla `videos` con un contador de reproducciones inicializado a 0. Antes de la inserción, verifica que no exista un vídeo con el mismo título.
  - `listar()`: Recupera todos los vídeos de la base de datos y los retorna como un `ArrayList<String[]>`, donde cada array contiene los campos: `id`, `titulo`, `autor`, `fecha_creacion`, `duracion`, `reproducciones`, `descripcion` y `formato`.

2.2. Controlador (`controller`)

Los controladores son **Servlets** anotados con `@WebServlet` que interceptan las peticiones HTTP, procesan la lógica de la aplicación y redirigen a las vistas correspondientes.

- `ServletUsuarios.java` (URL: `/ServletUsuarios`):
  Gestiona las acciones de registro y login de usuarios a través del método `doPost`. Utiliza un parámetro oculto `accion` para distinguir entre las operaciones:
  - `registro`: Valida que todos los campos estén presentes y que las contraseñas coincidan. Delega la inserción en `Usuario.registrar()` y redirige a `resultado.jsp` con el mensaje apropiado.
  - `login`: Valida los campos de usuario y contraseña. Si la autenticación es exitosa, crea una sesión HTTP (`HttpSession`) con el atributo `usuario` y redirige a `registroVid.jsp`. En caso de fallo, muestra el mensaje de error en `resultado.jsp`.
- `ServletVideos.java` (URL: `/ServletVideos`):
  Gestiona el registro y listado de vídeos, protegiendo ambas operaciones con verificación de sesión:
  - `doPost`: Recibe los datos del formulario de registro de vídeo, valida campos vacíos y parsea los campos de duración (horas, minutos, segundos) con control de `NumberFormatException`. Formatea la duración como `HH:MM:SS` y delega en `Video.registrar()`.

- `doGet`: Invoca `Video.listar()` para obtener el listado completo de vídeos y lo envía como atributo de request a `listaVideos.jsp`.

2.3. Vista (`view`)

Las vistas se implementan mediante **páginas JSP** ubicadas en `webapp/vista/`. Cada vista es responsable únicamente de la presentación de datos; no contiene lógica de negocio ni acceso a base de datos.

Vista	Función
`login.jsp`	Formulario de autenticación con campos usuario/contraseña. Incluye botón de navegación al registro de usuarios.
`registroUsu.jsp`	Formulario de registro de usuario con campos: nombre, apellidos, email, username, password y confirmación de password.
`registroVid.jsp`	Formulario de registro de vídeo con campos: título, autor, fecha, duración (horas/minutos/segundos), descripción y formato. Incluye navegación a la lista de vídeos y logout.
`listaVideos.jsp`	Muestra una tabla HTML con todos los vídeos registrados. Si no hay vídeos, muestra el mensaje «No hay videos registrados.» Incluye navegación al registro y logout.
`resultado.jsp`	Página genérica de resultado que muestra el mensaje recibido como atributo de request (éxito o error).

3 Gestión de Sesiones

La aplicación implementa un sistema completo de gestión de sesiones HTTP para controlar el acceso de los usuarios autenticados:

1. **Creación de la sesión**: Al realizar un login exitoso en `ServletUsuarios`, se crea una sesión HTTP mediante `request.getSession()` y se almacena el nombre de usuario con `session.setAttribute(usuario, username)`.

2. **Verificación de sesión en los Servlets**: Tanto `doPost` como `doGet` de `ServletVideos` verifican la existencia de una sesión válida antes de procesar cualquier petición:

```
HttpSession session = request.getSession(false);
if (session == null || session.getAttribute("usuario") ==
   null) {
    response.sendRedirect(request.getContextPath() + "/
       vista/login.jsp");
    return;
}
```

El uso de `getSession(false)` garantiza que no se cree una nueva sesión si no existe una previamente establecida.

3. **Verificación de sesión en las JSP**: Las vistas protegidas (`registroVid.jsp` y `listaVideos.jsp`) incluyen un bloque de scriptlet que verifica la sesión y redirige al login si el usuario no está autenticado.
4. **Logout**: Se implementa mediante la navegación directa a `login.jsp`, disponible en las vistas de registro de vídeos, lista de vídeos y resultado.

4 Gestión de la Base de Datos

4.1. Esquema de tablas

La aplicación utiliza dos tablas en la base de datos Apache Derby:

Tabla `usuarios`:

Columna	Descripción
`nombre`	Nombre del usuario
`apellidos`	Apellidos del usuario
`email`	Correo electrónico
`username`	Nombre de usuario (clave única)
`password`	Contraseña

Tabla `videos`:

Columna	Descripción
`id`	Identificador del vídeo (clave primaria)
`titulo`	Título del vídeo (único)
`autor`	Autor del vídeo
`fecha_creacion`	Fecha de creación (tipo `DATE`)
`duracion`	Duración en formato `HH:MM:SS` (tipo `VARCHAR`)
`reproducciones`	Número de reproducciones (tipo `INTEGER`, se inicializa a 0)
`descripcion`	Descripción del vídeo
`formato`	Formato del vídeo

4.2. Prevención de inyección SQL

Todas las consultas a la base de datos utilizan `PreparedStatement` con parámetros posicionales, lo que previene ataques de inyección SQL. No se construyen consultas SQL mediante concatenación de cadenas en ningún punto de la aplicación.

4.3. Gestión de conexiones

Las conexiones se obtienen mediante `DBConnection.getConnection()` y se cierran automáticamente gracias al uso de sentencias **try-with-resources** (`try (Connection conn = ...)`), garantizando la correcta liberación de recursos incluso en caso de excepciones.

5 Uso de Packages

El código Java está organizado en paquetes que reflejan las responsabilidades del patrón MVC:

- `model`: Contiene las clases de modelo y acceso a datos.
- `view`: Las vistas JSP se ubican en el directorio `webapp/vista/`.
- `controller`: Contiene los Servlets que actúan como controladores.

Esta separación garantiza una clara delimitación de responsabilidades y facilita el mantenimiento y la extensibilidad del código.

6 Control de Errores y Validación de Entrada

La aplicación implementa múltiples capas de validación y control de errores:

6.1. Validación en el lado del cliente (HTML)

Los formularios JSP utilizan el atributo `required` en los campos de entrada, lo que impide el envío de formularios con campos vacíos a nivel de navegador. Además, se emplean tipos de entrada apropiados:

- `type=.email"` para la validación del formato de correo electrónico.
- `type="date"` para la selección de fechas.
- `type="password"` para la ocultación de contraseñas.
- `type="number"` con atributos `min` y `max` para los campos de duración.

6.2. Validación en el lado del servidor

Independientemente de la validación del cliente, los Servlets verifican en el servidor que:

- Todos los campos obligatorios no sean `null`.
- Las contraseñas de registro coincidan (`password.equals(password2)`).
- Los valores de duración (horas, minutos, segundos) sean numéricos válidos, capturando `NumberFormatException`.

6.3. Control de errores en redirecciones

- Las operaciones exitosas y fallidas redirigen a `resultado.jsp` con un mensaje descriptivo mediante `request.setAttribute("mensaje", ...)` y `RequestDispatcher.forward()`.
- El acceso no autorizado (sin sesión) redirige automáticamente a `login.jsp` mediante `response.sendRedirect()`.
- Los errores de base de datos se capturan con bloques `try-catch`, evitando que excepciones no controladas lleguen al usuario.

6.4. Control de duplicados

- El registro de un usuario con un `username` ya existente retorna un mensaje de error.
- El registro de un vídeo con un `titulo` ya existente retorna un mensaje de error.

7 Configuración del Despliegue

El fichero `web.xml` define la configuración del descriptor de despliegue conforme a la especificación Jakarta EE Web Application 5.0:

```
<welcome-file-list>
    <welcome-file>vista/login.jsp</welcome-file>
</welcome-file-list>
```

Adicionalmente, el fichero `index.jsp` actúa como punto de entrada alternativo, redirigiendo automáticamente al formulario de login mediante `response.sendRedirect("vista/login.jsp")`.

Los Servlets se registran mediante la anotación `@WebServlet`, eliminando la necesidad de configurarlos explícitamente en `web.xml`.

8 Flujo de Navegación de la Aplicación

El siguiente diagrama describe el flujo de navegación principal de la aplicación:

Flujo de navegación

```
[index.jsp] --- redireccion ---> [login.jsp]
                                        |
                        +--------------+--------------+
                        |              |              |
                   Login exitoso  Login fallido   Ir a registro
                        |              |              |
                        v              v              v
               [registroVid.jsp] [resultado.jsp] [registroUsu.jsp]
                  |           |                         |
         Registrar video  Ver lista           Registrar usuario
                  |           |                         |
                  v           v                         v
         [resultado.jsp] [listaVideos.jsp]    [resultado.jsp]
```

- Todas las páginas protegidas (registro de vídeo, lista de vídeos) verifican la sesión y redirigen a `login.jsp` si el usuario no está autenticado.
- Desde cualquier página protegida es posible acceder al logout, que redirige al formulario de login.

9 Conclusiones

Este projecto implementa una aplicación web funcional de gestión de vídeos que cumple con los requisitos fundamentales del proyecto:

- **Patrón MVC correctamente implementado**, con separación clara entre modelo, vista y controlador.
- **Gestión completa de sesiones** HTTP para controlar el acceso de usuarios autenticados.
- **Acceso seguro a base de datos** mediante `PreparedStatement` y gestión adecuada de conexiones.
- **Validación de entrada** tanto en el cliente (atributos HTML) como en el servidor (verificaciones en los Servlets).
- **Control de errores** con mensajes informativos al usuario y captura de excepciones.
- **Organización del código** en paquetes Java que reflejan las responsabilidades de cada componente.

La aplicación constituye una base sólida que podría ser ampliada con funcionalidades adicionales como la edición y eliminación de vídeos, la implementación de paginación en el listado, o la incorporación de un sistema de roles de usuario.